COMP 7500 Advanced Operating Systems

Project 4: cpmFS - A Simple File System

Frequently Asked Questions

Document History: Created on April 25, 2019. Revised on April 24, 2023. Version 1.9

1. In cpmfsys.h file there are 5 functions at the bottom:
   - `cpmCopy`
   - `cpmOpen`
   - `cpmClose`
   - `cpmRead`
   - `cpmWrite`

   which says that need not to be implemented for project 4. Are they need to be implemented in project 4?

Answer: You don't need to implement the above five functions in project 4. Your source code file `cpmfsys.c` should include the implementations of the following nine functions:

- `mkDirStruct()`
- `writeDirStruct()`
- `makeFreeList()`
- `printFreeList()`
- `cpmDir()`
- `checkLegalName()`
- `findExtentWithName()`
- `cpmDelete()`
- `cpmRename()`

2. Are we supposed to use `diskSimulator.c` and other files from the course repository or are we supposed to build our own version of all of those files?

Answer: You only need to implement `cpmfsys.c` by making use of the existing `diskSimulator.c` and the other files from the course repository.

3. Do we ignore the actual execution time and use the execution time given by the run command? If the last sector is fully used, would it be save to assume that RC would have the

value of 8? This would indicate that all 1024 (8*128) bytes have been used. However, so far I do not see any RC with the value of 8.

Answer: RC is the number of sectors in the last partially-filled block. The maximum possible RC should be 7.

4. Regular expression aren't part of ANSI C. But I found that regex.h can do regular expression which can help us check the file name more easily. Can we use this header file?

Answer: You are allowed to include `regex.h` in your implementation.

5. When I try to inspect the disk image, it appears the only block with contents is block 0. Is this correct? Since block 0 refers to locations for file contents, I expected to find that actual content elsewhere in the disk image. It seems the project can be completed without having this content, but I was just checking to see if I was missing something.

Answer: You must exam the extents in block 0 to determine if there are allocated and used blocks (i.e., block address of 1-255). The information can be found in the last 16 bytes of each extent. I agree with you that this project might be completed without having the data content (i.e., blocks are allocated without containing any content).

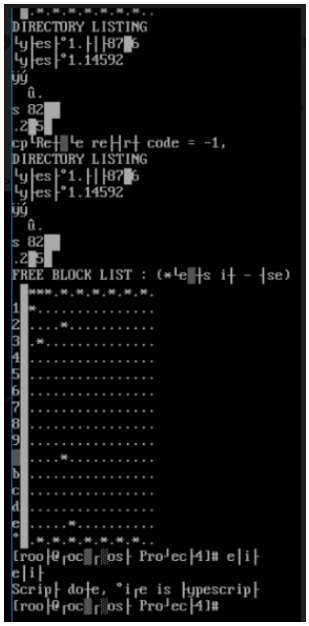
6. **Function cpmDir prints weired output.** It also caused the centOS system a lot of errored displays. The solutions I tried as follows:
   1) I tried to download the image1.img again, and the error still shows.
   2) If I only print the first 5 lines of image1.img (or the first two files), no error in the output.
   3) If I use script to save the output, centOS error display are correct in typescript file, but the weired output for the file names are still there.

   Does anyone have the same issue?

   In saved typescript:

```
DIRECTORY LISTING
mytestf1.txt 8706
mytestf1. tx 14592
ÿýû.
s 8200
01 . 2050
```

   Direct captured from centOS:



Answer:
(1) Your `cpmDir()` function might have a bug. Prior to printing an extent (say `d`) in block 0, your `cpmDir` must check the `status` of the extend (i.e., `d->status != 0xe5`). If `d->status == 0xe5`, then this extent is invalid. Don't print the extent if it is invalid.

(2) How did you display file names? A sample statement to print out file name and size is given below. We assume that you correctly calculate file size (i.e., file_size) prior to this statement.

```
printf("%s.%s %d\n", d->name, d->extension, file_size);
```

(3) I suggest you to use *free(<yourPointerNameInsideForLoop>),*

(4) Peijie Chen (Spring 2019) shared his programming experience with us as follows.
"I have encountered similar issues with you when I tried to use the tux machine (which is also a cent OS). But the problem disappears when I switched to windows and used cygwin64 to comply it. I used Visual studio code – VSCode - to write my code. It may or may not solve your problem, but if you think your implementation is correct, I recommend you give a try."

7. How can I make use of `blockRead()` to retrieve a block from image1.img? (Spring'20)

Answer: A sample usage below demonstrates how to use the `blockRead()` to read a block.

```
uint8_t dirBlock[BLOCK_SIZE];

/* read the directory block from the disk */
blockRead(dirBlock,(uint8_t) 0);
```

8. Initialization of disk. In project4->disksimulator.c-> for the initialization of disk[][], it is described in comments that the 1st index is the sector number but it has been told in the class that a block contains 8 sectors of 128 bytes each. Also, the disk is considered as 256 blocks of 1024 bytes each. Kindly confirm if the following initialization means that the disk has 256 blocks of 1024 bytes each? (Spring'20)

Answer: Yes, this is a static data structure of two-dimentional array of 256 blocks, each of which has 1024 bytes.

9. **Printblock.** In disksimulator.c, in printblock(), why are we checking after 16 bytes and why do we use %4x and %2x, when we don't know the type of information(datatype) that image contains? (Spring'20)

Answer: The "16 bytes" mean that the function prints out 16 bytes per line. Please test this function and sense the usage of printblock().

10. **File System Size.** If each image contains 256 blocks in total. Block 0 has 1024/32=32 extents with 16 block information each. That means 16(blocks per file extent) * 32(no of file extents in block 0) = 512 blocks are needed to fill the data whereas only 255 blocks are available in the image. Hence, I would like to ask how does a file system deal with this? (Spring'20)

Answer: You are right. If all the extents are fully utilized and all files contains 16 blocks, the file system will be running out of space. To fix this problem, we must extend the file system size by increasing the number of blocks from 256 to at least 513. You don't have to address this concern in this project.

11. **Freelist.** In freelist, it was mentioned that a bit is reserved in every block to know if it is used or not. Hence, I am a little confused about how to convert it into a freelist[]. (Spring'20)

Answer: You may define a boolean data type first. The data structure of all the element in the freelist[ ] array is boolean (e.g., True = free, False = allocated).

12. **File Names.** I am confused with some instructions of bool checkLegalName(char *name) function. In instruction "there does not have to be any valid character in the extension – it can all be blanks" But in checkLegalName function it says "returns true for legal name (8.3 format)". (Spring'20)

Answer: Let me give you a few examples below:

mytestf1 (legal or true)
mytestf1.txt (legal or true)
mytestf11 (illegal or false)
mytestf1.txtt (illegal or false)

13. **DirStructType.** In the Project Specification it says "If the name is less than 8 bytes, the remaining bytes are padded with blanks (ASCII 32). There is no terminating '\0' after the last valid character of the filename, just blanks"

    In lecture 33/handout it says "

```
If name length < 8, filled by ' '
obtain dir->name from in-memory block0
add '\0' at the end of dir->name
```

    The project specification also says that the file extension can be all blanks, but the lecture says to add a '\0' at the end. Which should I do? (Spring'20)

Answer: File extents (i.e., direcotry entries) are represented in two different formats. In block 0, each extent is 32 bytes in length. However, when you read an extent from bloc 0 into an DirStructType variable using the mkDirStruct() function, the DirStructType variable has 34 bytes rather than 32 bytes. This is because we must add '\0' into the DirStructType variable for the file name and extension name, respectively.

In a nutshell, there are the two formats:

- Format 1: All 32-byte extents in block 0 have no "\0" in file names and extension names.
- Format 2: All 34-byte DirStructType variables have "\0" attached at the end of file names and extension names.

14. **DirStructType.** The dirStruct definition is in cpmfsys.h. It has name defined as char name[9] and extension is defined as char extension[4], but the spec shows the name as being 8 characters long and the extension as being 3 characters long. The disk image matches the definition from the spec. I'm assuming there's an issue with the header file.

Answer: dirStruct is a data structure defined for in-memory block 0, which is slightly different from the in-disk block 0. The char array name[9] in dirStruct contains up to 8 chars and the terminator $\0$, which turns the char array into a string. Please refer to FAQ question 13 for more details.

15. **Makefile.** The makefile doesn't specify the c99 flag necessary to declare a variable inside of a for loop. Are we able to submit our own makefile with the c99 flag specified?

Answer: Yes, you should modify the makefile by specifying the c99 flag. Please submit a makefile that results in successful compilation of your code on the machines in the Linux lab: http://www.eng.auburn.edu/admin/ens/helpdesk/off-campus/access-information.html

16. **Image1.img.** What method should be used to view image1.img file? (Spring'21. Contributed by Mahmoud Abdalkarim)

Answer: "`hexdump -C image1.img`" in the command line of your Linux terminal.

17. **File Name.** For functions that take at least one file name as a parameter (i.e., checkLegalName, findExtentWithName, cpmDelete, and cpmRename), is the extension (if there is one) part of the passed file name? Also, is there a period right after the file name? (Spring'22)

Answer: If a function takes a file name (one parameter) as an input, the name embraces (1) a file name (8 chars), (2) an extension name (3 chars), and (3) a period ('.') siting between the file name and the extension name. Your function is responsible for parsing the full name into these three pieces.

18. **cpmDelete Return Value.** The function cpmDelete returns an int. Is this int supposed to reflect the status of the delete? ex (-1 for unsuccessful, non negative for successful) Or is the return value supposed to be the index of the directory deleted? (positive number = index, negative number for unsuccessful delete)? (Spring'22)

Answer: There can be a few possibilities:

- file not exist
- successfully deleted
- other errors, maybe

It is acceptable to return "-1" for files not found or any other errors.

19. **cpmDir Implementation.** I followed the sample code snippet in '32-Project 4-2 cpmFS Design and Algorithms.pptx', which is

    cpm_block0 = blockRead(block0);

I translated in my code to be:

```
uint8_t cpm_block0 = blockRead(buffer,0);
```

Is this the correct implementation? (Spring'22)

Answer: No, you didn't correctly use the blockRead() function because this function return 0 as success while storing block0 in the first parameter. The correct implementation should be:

```
blockRead(cmp_block0, (uint8_t) 0);
```

20. **Valid Filename Specifics.** When determining the validity of a filename, the instructions state that only the first character is checked for validity. Does this mean we need to account for other symbols that are typically legal in the rest of the filename, or just assume that they will all be alphanumeric only? (Spring'22)

Answer: You still need to check if each character in file names and extension names are legal. The valid characters should be either upper/lower case letters (A-Z, a-z) or numbers 0-9.

21. **File Name and Period.** Does a period follow the file name only if the corresponding file has an extension? (Spring'22)

Answer: Yes, a period is needed only if there is an extension name.

22. **Rename "mytestf." to "mytestv2.x".** It shows "cpmRename("mytestf","mytestv2.x")" on the source code "fsysdriver.c". However, image1.img list result is shown as "mytestf." whose extension is blank. I am not sure should I treat them as the same file or treat them as different. Because from the "sampleOutput.txt". They are treated as the same file and return code = 0, if I treat them as different files, the return code = -1. Does a period follow the file name only if the corresponding file has an extension? (Spring'22)

Answer: Let's assume we don't intent to modify the driver code fsysdriver.c, you may update your comparison method, which does not count '.'.

23. **User Codes in Image1.img.** I have implemented the mkDirStruct() and cpmDir() methods and I can successfully print out the dir structure with the file names, extensions and file size. There is some formatting to I need to fix which should be straight forward.

    Since the recommended way is to have a condition to see if the extent is used or not, I've added this if condition:

if(cpm_dir->status >=0), then print the details

But currently it prints all the extents in block 0 regardless if it has file metadata or not and below is the output. If you see the output I'm printing out the user code at the end of the line just to show the user code I'm getting for each extent.

Note that even the extents that has metadata of files and the ones that don't have the user code set to '0' looks like except for one which is 16.

I was expecting that if the extent is used then the user code will be between 0 and 15 as mentioned in section 3.2 in the proj 4 specification document, and if the extent is Not used it will have a different value as mentioned.

So I'm wondering if I'm missing something here in my code or is the data in the extents in Block 0 in the given image1.img correct or not?

```
DIRECTORY LISTING
mytestf.1tx 15745 bytes |---- user status is: 0
holefil.etx 2433 bytes |---- user status is: 16
\345. -1024 bytes |---- user status is: 0
shortf . ps 1409 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
mytestf.    1409 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
\345. -1024 bytes |---- user status is: 0
```

Answer: This bug is likely to be related to the malloc() function of the buffer and blockRead() within the loop of cmpDir. You must, at the end of each iteration, free the buffer before call the malloc() again. If you move the buffer pointer, the status may not point to the right place and therefore not have the right value.

24. **Source Modification.** I'm see that we're only supposed to submit our `cpmfsys.c` source file, leaving the other files unchanged. I had also modified `cpmfsys.h', renaming some parameters for my clarity, adding a few extra function declarations, and tweaking the struct slightly to work better with my functions. Is it permissible to include a modified header file in our submission, and if not how should we add extra functions and meet guidelines on code readability?

Answer: You cannot modify cpmfsys.h file to add any new features because the only file you need to submit is cpmfsys.c. If you must add extra supporting functions, you should include these functions' prototypes as well as implementations in cpmfsys.c.

25. **Design of project 4 and DFD.** From my perspective nearly all the design was done on our behalf, manifested in the from of the provided files, simply leaving the implementation to us. So if I could please get some additional clarification of what is expected for the design portion of the report, as well as the for the DFD, I would appreciate it.

    The project specifications say that the report should contain "The design of your cpmFS." but considering we just implemented pre-defined functions I am unsure of what my design contributions would constitute.

    The project rubric on canvas mentions additional items: data flow diagram, function prototypes, and discussions on design issues. Again, considering that most of the project was provided to us, I am unsure of what I am meant to provide for these items.

Answer: You should draw your own DFD according to your understanding of the cpmFS design. Similarly, you must explain the function prototypes in your design document.